Universidade do Minho
Escola de Engenharia

2025/2026

Universidade do Minho - Escola de Engenharia

Sistemas Operativos

Gonçalo Paiva
a112257

João Fernandes
A103568

Paulo Silva
a109577

Índice

1 Runner ... **3**
1.1 Estrutura Geral ... 3
1.1.1 Execução de comandos -e ... 3
1.2 Consulta de Comandos -c ... 4
1.2.1 Encerramento do controller -s ... 4

2 Controller ... **5**
2.1 Gestão de concorrência e sincronização ... 5
2.1.1 Comunicação através de FIFOs ... 5
2.2 Estruturas internas de controlo ... 5
2.3 Ciclo principal do controller ... 5
2.3.1 Estruturas utilizadas ... 6
2.4 Políticas de escalonamento ... 6
2.4.1 FIFO (First In First Out) ... 6
2.4.2 Round Robin por utilizador ... 6
2.4.3 Implementação do Round Robin ... 7
2.5 Monitorização e logging ... 7
2.6 Consulta do estado do sistema ... 7
2.7 Terminação controlada ... 7
2.8 Modelo de funcionamento ... 7

3 Execução de comandos e suporte a operadores ... **8**
3.1 Estrutura interna do executor ... 8
3.2 Execução de comandos simples ... 8
3.3 Parsing de comandos ... 8
3.4 Suporte a redirecionamentos ... 9
3.5 Suporte a pipes ... 9

4 Testes realizados ... **10**
4.1 Testes funcionais ... 10
4.2 Testes de paralelização ... 10
4.3 Testes de políticas de escalonamento ... 10

5 Análise dos resultados ... **12**

6 Conclusão ... **12**

Lista de Figuras

Figura 1 Codigo na main do runner. 3
Figura 2 Exemplo de pipeline com múltiplos processos ligados por pipes. 9
Figura 3 Exemplo de escalonamento com política Round Robin. 11

1 Runner

O programa runner é responsável por intermediar a execução de comandos submetidos pelos utilizadores. Antes de executar qualquer comando, o runner comunica com o controller através de pipes com nome (FIFOs), aguardando autorização de execução.

O programa suporta três modos de funcionamento:

execução de comandos (-e); consulta de comandos (-c); terminação do controller (-s).

1.1 Estrutura Geral

O programa começa por validar os argumentos recebidos na linha de comandos e encaminha a execução para o modo correspondente.

```
int main(int argc, char* argv[]){

    if(argc == 4 && strcmp(argv[1],"-e")==0)
        modo_exec(argv);
    else if (argc == 2 && strcmp(argv[1], "-c") == 0)
        modo_consulta();
    else if (argc == 2 && strcmp(argv[1], "-s") == 0)
        modo_terminar();
    else {
        write(2,"Invalid option or arguments\n",15);
        return 1;
    }
```

Figura 1: Codigo na main do runner.

Esta validação inicial garante que o `runner` apenas é executado com parâmetros válidos, evitando comportamentos indefinidos ou mensagens mal formadas para o `controller`.

A comunicação entre processos é feita utilizando pipes com nome (FIFOs). Cada runner cria uma FIFO privada associada ao seu PID, permitindo ao controller responder individualmente a cada pedido.

1.1.1 Execução de comandos -e

No modo de execução (-e), o runner recebe o identificador do utilizador e o comando a executar. Inicialmente, é criada uma FIFO privada para comunicação com o controller.

```
sprintf(mypipe, FIFO_RUNNER, runner_pid);
unlink(mypipe);
mkfifo(mypipe, 0666);
```

De seguida, o runner constrói uma mensagem contendo:

tipo de operação; PID do processo; identificador do utilizador; comando a executar.

O pedido é enviado ao controller através da FIFO principal:

```
enviar_pedido(msg);
```

Depois de enviar o pedido de execução, o `runner` não executa imediatamente o comando. Em vez disso, fica bloqueado à espera de autorização explícita do `controller`.

A sincronização é feita através da FIFO privada do `runner`:

```
esperar_autorizacao(msg, mypipe);
```

Este mecanismo garante que o número máximo de comandos em execução simultânea nunca ultrapassa o limite definido no arranque do `controller`.

Só depois de receber uma resposta positiva (`autorizado = 1`) é que o `runner` chama o módulo `executor` e inicia efetivamente a execução do comando.

Quando autorizado, o comando é executado a partir da função executar_linha_comando(msg, comando), a forma como a função executa os diferentes comandos e como ele suporta os diferentes operadores de redirecionamento e de comunicação entre processos vai ser explicada mais adiante no relatório.

Após o término do comando, o runner notifica o controller da conclusão da execução:

```
notificar_concluido(msg);
```

1.2 Consulta de Comandos -c

O modo de consulta (-c) permite ao utilizador obter informação sobre os comandos atualmente em execução e em espera.

Tal como nos restantes modos, o runner começa por criar uma FIFO privada para receber a resposta do controller.

```
mkfifo(mypipe, 0666);
```

De seguida, é enviada uma mensagem de consulta:

```
Msg msg = fazer_Msg('c', runner_pid, "", "");
```

O controller processa o pedido e envia a informação solicitada para a FIFO privada do runner.

A leitura da resposta é feita da seguinte forma:

```
while ((n = read(fd2, buffer, sizeof(buffer))) > 0) {
    write(1, buffer, n);
}
```

O runner limita-se a apresentar no stdout a informação recebida, sendo todo o processamento realizado pelo controller.

1.2.1 Encerramento do controller -s

O modo -s é utilizado para solicitar o encerramento do controller.

O runner inicialmente envia uma mensagem de encerramento para a pipe do controler:

```
Msg msg = fazer_Msg('s', runner_pid, "", "");
```

Após o envio do pedido, o runner permanece bloqueado à espera de confirmação de encerramento.

O controller apenas termina depois de todos os comandos em execução serem concluídos. Quando o encerramento é finalizado, o runner recebe uma resposta de confirmação e informa o utilizador através do stdout.

2 Controller

O `controller` é o componente central do sistema e atua como um escalonador de comandos concorrentes. É responsável por receber pedidos de múltiplos `runners` e coordenar a sua execução de forma centralizada. Não executa os comandos diretamente, funcionando apenas como um gestor de autorização, escalonamento e controlo de concorrência.

2.1 Gestão de concorrência e sincronização

O controller implementa um modelo de controlo de concorrência baseado num limite máximo de execuções simultâneas definido no arranque do programa. Sempre que recebe novos pedidos, decide se estes podem ser executados imediatamente ou se devem ser colocados em espera, garantindo assim que o sistema não ultrapassa a capacidade definida.

Desta forma, o controller assegura também a sincronização entre múltiplos `runners`, organizando os pedidos de forma consistente e evitando conflitos na execução concorrente.

2.1.1 Comunicação através de FIFOs

A comunicação entre `runner` e `controller` foi implementada recorrendo a pipes com nome (`FIFOs`), permitindo comunicação entre processos independentes sem necessidade de memória partilhada.

O `controller` cria uma FIFO principal conhecida por todos os `runners`:

```
mkfifo(FIFO_CONTROLLER, 0600);
```

Todos os pedidos são enviados através desta FIFO principal. No entanto, como o `controller` necessita responder individualmente a cada `runner`, cada processo cliente cria também uma FIFO privada associada ao seu PID.

O nome da FIFO privada é gerado dinamicamente:

```
sprintf(mypipe, FIFO_RUNNER, runner_pid);
```

Desta forma, o `controller` consegue comunicar diretamente com o `runner` correto sem interferir com os restantes processos.

Esta solução aproxima-se do modelo cliente-servidor clássico, onde o `controller` atua como servidor central e os `runners` funcionam como clientes independentes.

2.2 Estruturas internas de controlo

Para conseguir gerir o estado do sistema, o controller mantém informação sobre os comandos em execução e os comandos em espera. Esta informação é armazenada em estruturas em memória que permitem acompanhar o ciclo de vida de cada comando desde a sua submissão até à conclusão.

Estas estruturas tornam possível saber em qualquer momento quais os comandos que estão a ser executados, quais estão em fila de espera e quantos slots de execução ainda estão disponíveis.

2.3 Ciclo principal do controller

O funcionamento do `controller` baseia-se num ciclo infinito responsável por receber e tratar mensagens enviadas pelos `runners`.

A leitura é feita através da chamada:

```
while(read(fifo_server, &msg, sizeof(Msg)) > 0)
```

Cada mensagem recebida contém:

- o tipo de operação;
- o PID do `runner`;

- o identificador do utilizador;
- o comando associado.

Dependendo do valor de `msg.operacao`, o controller executa diferentes ações:

- `'e'` — pedido de execução;
- `'c'` — consulta do estado atual;
- `'f'` — notificação de conclusão;
- `'s'` — pedido de encerramento.

Esta abordagem baseada em mensagens simplificou bastante a organização do programa, já que todas as interações passam por um único ponto central de processamento.

2.3.1 Estruturas utilizadas

O estado interno do `controller` é mantido recorrendo a estruturas simples em memória estática. Esta abordagem foi escolhida por ser suficiente para a dimensão do problema e por permitir um controlo mais direto sobre os comandos em execução e em espera.

Para representar cada comando foi utilizada a estrutura `Comando`, onde são armazenados:

- o PID do `runner`;
- o identificador do utilizador;
- o comando submetido;
- o instante de início da execução.

A informação temporal é obtida através da função `gettimeofday()`, permitindo calcular posteriormente a duração total da execução para efeitos de logging.

Os comandos em espera são armazenados no vetor:

```
Comando fila_espera[100];
```

Já os comandos atualmente em execução são guardados em:

```
Comando em_exec[100];
```

O uso de vetores simplificou bastante a implementação das operações de inserção, remoção e consulta, uma vez que o número máximo de comandos esperado para os testes do projeto era reduzido.

Além disso, o controller mantém variáveis auxiliares importantes para o funcionamento do sistema:

- `em_execucao` — número atual de comandos em execução;
- `fila_size` — número de comandos em espera;
- `em_exec_size` — número de comandos ativos;
- `ultimo_user` — utilizado pela política Round Robin;
- `shutdown` — indica se foi pedido o encerramento do sistema.

2.4 Políticas de escalonamento

O sistema suporta duas políticas de escalonamento:

2.4.1 FIFO (First In First Out)

Os comandos são executados pela ordem de chegada, garantindo um comportamento simples e previsível.

2.4.2 Round Robin por utilizador

A política tenta alternar entre utilizadores diferentes, evitando que um único utilizador monopolize o sistema. Esta abordagem promove maior justiça na distribuição dos recursos.

2.4.3 Implementação do Round Robin

A política Round Robin foi implementada de forma simplificada, alternando entre utilizadores diferentes sempre que existe mais do que um utilizador na fila de espera.

Para isso, o `controller` guarda o identificador do último utilizador servido na variável:

```
char ultimo_user[32];
```

Quando um novo slot de execução fica disponível, o controller percorre a fila de espera à procura do primeiro comando pertencente a um utilizador diferente de `ultimo_user`.

Caso exista um utilizador diferente, esse comando é autorizado primeiro. Caso contrário, o comportamento degrada-se automaticamente para FIFO.

Esta solução permitiu implementar uma política mais justa sem necessidade de estruturas de dados complexas nem múltiplas filas por utilizador.

A política é definida no arranque do `controller`.

2.5 Monitorização e logging

O controller também é responsável por registar informação sobre cada execução concluída. Para cada comando executado são guardados o identificador do utilizador, o comando executado e a duração total da execução.

Este registo é persistente e permite posteriormente analisar o comportamento do sistema, bem como avaliar o impacto das diferentes políticas de escalonamento implementadas.

2.6 Consulta do estado do sistema

O controller permite ainda que os `runners` consultem o estado atual do sistema. Esta consulta devolve informação sobre os comandos em execução e os comandos em espera, permitindo ao utilizador ter visibilidade sobre o estado global do sistema sem interferir com a sua execução.

2.7 Terminação controlada

O processo de terminação do controller é feito de forma controlada. Embora possa ser solicitado por um `runner`, o controller apenas termina quando já não existem comandos em execução nem em espera, garantindo assim que todas as tarefas são concluídas antes do encerramento do sistema.

2.8 Modelo de funcionamento

O controller segue um modelo centralizado baseado em eventos. Recebe continuamente pedidos através de FIFOs, atualiza o seu estado interno, decide o escalonamento dos comandos e comunica com os `runners` através de respostas apropriadas.

Este modelo permite gerir concorrência de forma eficiente sem necessidade de múltiplos threads no próprio controller.

3 Execução de comandos e suporte a operadores

Para tratar da execução dos comandos submetidos pelos utilizadores, foi criado um módulo auxiliar chamado `executor`. Este módulo é chamado pelo `runner` depois de o `controller` autorizar a execução do comando. Desta forma, o `runner` fica mais focado na comunicação com o `controller`, enquanto o `executor` trata apenas da parte da execução dos comandos. Esta separação acabou por tornar o código mais organizado, mais fácil de manter e também mais simples de compreender.

O `executor` foi desenvolvido de forma modular, separando claramente as fases de parsing, aplicação de redirecionamentos e execução final, o que permite reutilizar e isolar funcionalidades específicas do processamento de comandos.

3.1 Estrutura interna do executor

Antes da execução de qualquer comando, é utilizada uma estrutura, que guarda toda a informação necessária para a execução:

- argumentos do comando (`argv`);
- ficheiro de entrada (`input_file`);
- ficheiro de saída (`output_file`);
- ficheiro de erro (`error_file`).

Esta estrutura permite representar de forma uniforme tanto comandos simples como comandos com redirecionamentos.

Para garantir um estado limpo antes de cada parsing, é utilizada uma função de inicialização que coloca todos os campos a `NULL`, evitando lixo de memória e comportamentos indefinidos.

Além disso, a linha de comando é previamente tratada para remover o caracter de newline (`\n`), garantindo consistência no parsing.

3.2 Execução de comandos simples

Numa primeira fase, começou-se por implementar a execução de comandos simples, ou seja, comandos sem redirecionamentos nem pipes. A linha de comando recebida é dividida em argumentos através de `strtok()`, sendo posteriormente passada para `execvp()`.

O processo de execução segue o modelo clássico Unix:

- criação de um processo filho com `fork()`;
- no filho, execução do comando com `execvp()`;
- no processo pai, espera pelo término com `waitpid()`.

Em caso de erro no `fork()` ou no `execvp()`, o processo termina imediatamente com `_exit(1)`, garantindo que não existem processos órfãos ou comportamentos inconsistentes.

3.3 Parsing de comandos

O parsing de cada comando simples é feito manualmente, identificando tanto argumentos como operadores especiais.

Durante este processo são tratados três tipos de redirecionamento:

- `<` para entrada;
- `>` para saída;
- `2>` para erros.

Os operadores não são guardados no vetor de argumentos, sendo apenas armazenados os ficheiros associados. Isto garante compatibilidade direta com `execvp()`.

Se algum operador não tiver ficheiro associado, o comando é considerado inválido.

3.4 Suporte a redirecionamentos

Depois disso, foi acrescentado suporte aos operadores de redirecionamento pedidos no trabalho.

Antes da execução do comando no processo filho, os ficheiros associados são abertos com `open()` e ligados aos descritores standard com `dup2()`:

- `stdin` no caso de <;
- `stdout` no caso de >;
- `stderr` no caso de 2>.

Com isto, passou a ser possível executar corretamente comandos com leitura a partir de ficheiros, escrita para ficheiros e redirecionamento de erros.

3.5 Suporte a pipes

Numa fase seguinte foi implementado o suporte ao operador `|`, permitindo executar pipelines de comandos.

A linha de comando é inicialmente dividida em várias partes através de `strtok()` com o delimitador `|`. Cada uma dessas partes representa um comando independente.

Para executar a pipeline:

- são criados pipes anónimos com `pipe()`;
- é criado um processo por cada comando;
- cada processo liga a sua entrada/saída aos pipes correspondentes com `dup2()`.

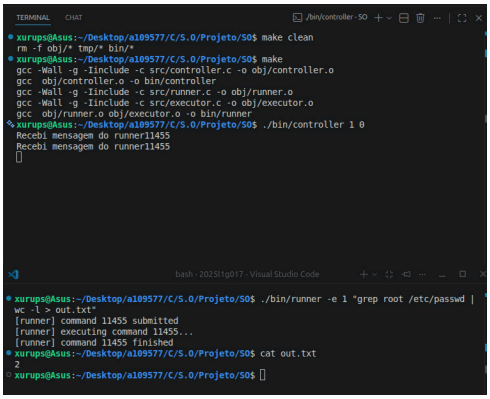

Figura 2: Exemplo de pipeline com múltiplos processos ligados por pipes.

O primeiro comando escreve para o pipe, o último lê do pipe anterior, e os comandos intermédios ligam ambos os lados.

Esta solução suporta tanto pipelines simples como pipelines com vários comandos, permitindo também combinar pipes com redirecionamentos.

4 Testes realizados

Para verificar se a solução estava a funcionar corretamente, foram criados vários scripts de teste, organizados na pasta `tests/`. Estes testes foram divididos em três grupos principais:

- testes funcionais do módulo de execução;
- testes de paralelização;
- testes de políticas de escalonamento.

4.1 Testes funcionais

Os testes funcionais serviram para confirmar se o módulo `executor` estava a tratar corretamente os vários casos previstos no enunciado. Foram testados:

- comandos simples;
- redirecionamento de saída com `>`;
- redirecionamento de erro com `2>`;
- redirecionamento de entrada com `<`;
- pipes simples com `|`;
- combinação de pipes com redirecionamentos;
- pipelines com mais de dois comandos;
- comandos inválidos e ficheiros inexistentes.

De forma geral, os resultados mostraram que os operadores pedidos no trabalho foram implementados com sucesso, incluindo situações em que aparecem combinados no mesmo comando.

4.2 Testes de paralelização

Para avaliar o impacto do número máximo de comandos em execução simultânea, foi utilizado um conjunto de quatro comandos `sleep 5`, variando o valor de `parallel-commands` no arranque do `controller`.

Configuração do controller	Carga de teste	Tempo total observado	Interpretação
1	4 comandos `sleep 5`	20 s	Execução praticamente sequencial
2	4 comandos `sleep 5`	10 s	Execução em dois blocos, com dois comandos em paralelo
4	4 comandos `sleep 5`	5 s	Execução dos quatro comandos em simultâneo

Tabela 1: Testes de paralelização com diferentes configurações do controller.

Como se pode observar, o aumento do grau de paralelismo reduziu de forma clara o tempo total de execução. Isto mostra que a configuração de `parallel-commands` influencia diretamente o desempenho do sistema e o tempo de espera dos utilizadores.

4.3 Testes de políticas de escalonamento

Por fim, foram realizados testes para comparar as duas políticas de escalonamento implementadas no `controller`: FIFO e Round Robin.

Para isso, foi usado um cenário com três comandos longos do utilizador 1 (`sleep 5`) e um comando mais curto do utilizador 2 (`sleep 1`).

No caso da política FIFO, os comandos foram executados pela ordem em que chegaram, o que fez com que o comando do utilizador 2 ficasse para o fim da fila.

Já na política Round Robin, o `controller` tentou alternar entre utilizadores, permitindo que o comando do utilizador 2 fosse executado mais cedo.

Este teste mostrou, por isso, que a política Round Robin distribui melhor a execução entre utilizadores e evita que um só utilizador ocupe a fila durante demasiado tempo.

Política	Configuração	Tempo total	Comportamento observado
FIFO	1	16 s	O comando do utilizador 2 ficou para o fim da fila.
Round Robin	1	16 s	O comando do utilizador 2 foi executado logo após o primeiro comando do utilizador 1.

Tabela 2: Comparação entre FIFO e Round Robin no cenário com 3 comandos `sleep 5` do utilizador 1 e 1 comando `sleep 1` do utilizador 2.

Como se pode observar, o tempo total de execução foi semelhante nos dois casos, uma vez que a carga total de trabalho executada foi a mesma. No entanto, a diferença tornou-se visível no tempo de resposta do utilizador 2.

Na política FIFO, o comando desse utilizador ficou para o fim da fila, enquanto em Round Robin foi executado logo após a conclusão do primeiro comando do utilizador 1.

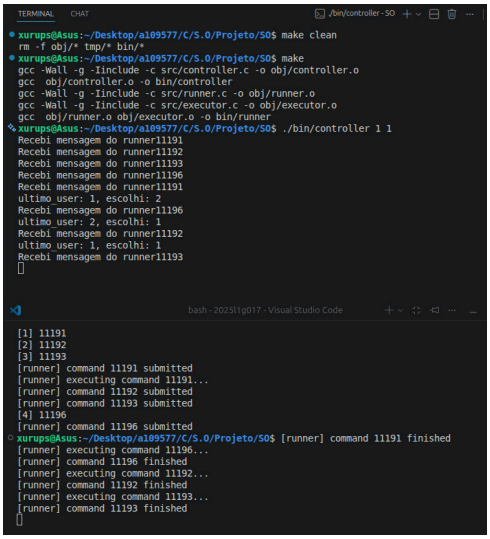

Figura 3: Exemplo de escalonamento com política Round Robin.

Como se pode verificar na figura anterior, o comando do utilizador 2 é efetivamente escalonado antes dos restantes comandos do utilizador 1, o que ilustra o comportamento mais justo da política Round Robin.

Assim, apesar de o tempo total do sistema não variar de forma significativa, a distribuição da execução entre utilizadores torna-se mais equilibrada.

5 Análise dos resultados

Através dos testes realizados foi possível confirmar que a solução suporta corretamente as principais funcionalidades pedidas no enunciado, sobretudo no que toca à execução de comandos, aos redirecionamentos e às pipelines.

O módulo `executor` conseguiu tratar tanto comandos simples como comandos mais completos, com vários operadores combinados.

Relativamente à paralelização, verificou-se que o número máximo de comandos em execução ao mesmo tempo influencia diretamente o desempenho do sistema. Nos testes com quatro comandos iguais, observou-se uma redução muito clara do tempo total à medida que aumentava o valor de `parallel-commands`.

Isto mostra que esta configuração tem impacto real na forma como os utilizadores sentem o sistema no uso prático.

No caso das políticas de escalonamento, os testes mostraram que FIFO e Round Robin podem ter tempos totais semelhantes quando a carga total de trabalho é a mesma.

No entanto, o Round Robin revelou-se mais justo, porque permitiu reduzir o tempo de espera de utilizadores com menos comandos submetidos.

Assim, mesmo que o tempo total do sistema não mude muito, a distribuição da execução entre utilizadores torna-se mais equilibrada.

6 Conclusão

Ao longo deste projeto foi desenvolvida uma solução concorrente baseada num modelo cliente-servidor, composta pelos programas `runner`, `controller` e pelo módulo auxiliar `executor`.

A solução implementada permitiu suportar corretamente a execução de comandos `bash`, incluindo funcionalidades como redirecionamentos de entrada e saída, pipes e pipelines com múltiplos comandos.

Além disso, o sistema conseguiu gerir múltiplos pedidos concorrentes através de um mecanismo de sincronização baseado em FIFOs, garantindo que o número máximo de comandos em execução simultânea respeita sempre a configuração definida no `controller`.

Também foram implementadas e comparadas duas políticas de escalonamento distintas: FIFO e Round Robin. Os testes realizados mostraram que, embora ambas apresentem desempenhos semelhantes em termos de tempo total de execução, a política Round Robin proporciona uma distribuição mais equilibrada dos recursos entre utilizadores.

De forma geral, considera-se que os objetivos principais do projeto foram cumpridos com sucesso, tendo sido construída uma solução funcional, modular e relativamente próxima do funcionamento básico de um sistema de gestão de execução de comandos `bash`.